Phishing URL Detection Using a Character-Level Convolutional Neural Network

Deep Learning Semester Project Report

Afzal Shah | Roll No. SU92-BSCSM-F23-521 | Section 7A

Date: 31 October 2026

Abstract

Phishing websites imitate trusted services in order to steal passwords, card numbers and other private data. Many attacks begin with a link, so a model that can judge a URL before the page is opened can give users an early warning. This project builds a character-level Convolutional Neural Network (CNN) that reads a URL one character at a time and predicts whether it is **phishing** or **legitimate**. The model was trained on a balanced public dataset of 340,000 URLs and reached **96.94% accuracy**, 97.24% precision, 96.62% recall, an F1-score of 96.93% and a ROC-AUC of 0.9965 on 33,973 test URLs that were never used for training or model selection. The network has only 17,633 parameters (about 69 KB), which made it possible to deploy it as a lightweight web application on Vercel with a live prediction demo and a public API. The complete source code, trained weights and tests are available in a public GitHub repository.

1. Introduction and Problem Statement

Phishing is a social-engineering attack in which a criminal sends a message containing a link to a fake website, for example a copy of a bank or payment login page. The victim enters credentials, and the attacker collects them. Blacklists of known phishing sites are useful, but new phishing URLs appear every day, so a blacklist is always behind. A learned model can recognise suspicious patterns in the text of a URL, such as misleading keywords (`paypal` , `secure` , `verify`), raw IP addresses, long random paths and unusual symbols, even for URLs it has never seen.

Problem statement. Given a URL string, classify it into one of two classes: legitimate (0) or phishing (1).

Objectives

1. Select and clean a suitable public dataset of labelled URLs.

2. Design and train a deep learning model that classifies URLs directly from their characters, without hand-crafted features.
3. Evaluate the model honestly on a held-out test set.
4. Save the trained weights and publish the code in a public GitHub repository.
5. Deploy a web application that lets anyone test the model with their own URLs.

2. Dataset

Item	Description
Source	Zenodo record 19371661 (https://zenodo.org/records/19371661), file `balanced_urls.csv`
Type	Secondary dataset, downloaded from a public repository
Size	340,000 rows, 2 columns (`label` , `url`)
Classes	170,000 legitimate and 170,000 phishing (perfectly balanced)
Missing values	None
URL length	Minimum 4, median 66, mean about 79.5, maximum 2,337 characters (standard deviation about 56.5)

The license and redistribution terms are those shown on the Zenodo record. For this reason the full CSV file is not copied into the GitHub repository; the repository links to the source instead.

2.1 Data cleaning

The data was inspected for missing values, empty strings, leading or trailing spaces and duplicates.

- No missing or empty URLs and no URLs with extra spaces were found.
- 277 extra rows were exact duplicate URLs (145 duplicate groups covering 422 rows). None of the duplicated URLs had conflicting labels, so one copy of each was kept.

- After removing duplicates, **339,723 URLs** remained: 170,000 legitimate and 169,723 phishing. The classes stay almost perfectly balanced, so accuracy is a meaningful metric here and no class weighting was needed.
- Labels were mapped to numbers: legitimate = 0, phishing = 1.

Removing duplicates matters because the same URL appearing in both training and test data would make the test score look better than it really is.

2.2 Train, validation and test split

The cleaned data was split with stratification (so every part keeps the same class ratio) and a fixed random seed of 42:

Part	URLs	Purpose
Training	271,777	Learning the model weights
Validation	33,973	Early stopping and choosing the best checkpoint
Test	33,973	Final, unbiased evaluation

The split is about 80% / 10% / 10%. The code verified that no URL appears in more than one part.

3. Preprocessing

A URL is not natural language, and it often contains random strings, numbers and symbols, so word-level tokenisation works poorly. Instead the model works at the **character level**.

1. **Vectorisation.** A Keras `TextVectorization` layer splits each URL into single characters (`split="character"`) with no lower-casing or other standardisation (`standardize=None`), so upper-case letters and symbols are preserved.
2. **Vocabulary.** The vocabulary was learned from the training split only, to avoid leaking information from validation or test data. It contains 97 entries: a padding token (index 0), an unknown-character token (index 1) and 95 characters seen in the training URLs (letters, digits and punctuation such as `/`, `.`, `:`, `-`, `=`, `%`, `?`, `&`, `_`).
3. **Fixed length.** Every URL is converted to exactly 300 integers. Longer URLs are cut after 300 characters and shorter ones are padded with zeros. Since the median URL has 66 characters, 300 covers almost all URLs completely.

The vocabulary and settings are saved in `vocabulary.json` and `preprocessing_config.json`, so the web application encodes URLs in exactly the same way as during training.

4. Model Architecture

A one-dimensional CNN was chosen because phishing indicators are local patterns inside the string (for example `login`, `verify`, `.php?cmd=`, or a long run of digits), and convolution filters are good at detecting such patterns wherever they appear.

Layer	Output shape	Parameters
Input (integer character indices)	(300)	0
Embedding (97 symbols to 32 dimensions)	(300, 32)	3,104
Conv1D (64 filters, kernel size 5, ReLU, same padding)	(300, 64)	10,304
Global Max Pooling 1D	(64)	0
Dense (64 units, ReLU)	(64)	4,160
Dropout (rate 0.3)	(64)	0
Dense (1 unit, sigmoid)	(1)	65
Total		**17,633 (about 69 KB)**

How it works. The embedding layer turns each character into a 32-number vector. The convolution layer slides 64 filters over groups of 5 neighbouring characters and responds to useful patterns. Global max pooling keeps, for each filter, the strongest response anywhere in the URL, which also makes the model independent of where in the URL the pattern occurs. The dense layer combines these 64 signals, dropout reduces overfitting, and the final sigmoid unit outputs the probability that the URL is phishing. A URL is labelled phishing when this probability is at least 0.5.

Design note. The first version used `mask_zero=True` in the embedding layer. Keras warned that the Conv1D layer does not support masks and would discard the mask information, so the model was rebuilt with `mask_zero=False`. Padding therefore passes through the network as an ordinary symbol, which the model learns to ignore.

5. Training

Setting	Value
Framework	TensorFlow 2.21 / Keras, Google Colab (GPU)
Optimizer	Adam, learning rate 0.001
Loss	Binary cross-entropy
Batch size	256
Maximum epochs	10
Early stopping	Monitor validation loss, patience 2, restore best weights
Checkpointing	Save the weights with the lowest validation loss
Metrics tracked	Accuracy, precision, recall
Logging	Per-epoch history saved to `training_history.csv`

Training stopped by early stopping after **6 epochs**. Epoch 1 reached 92.84% training and 95.64% validation accuracy, epoch 2 reached 96.11% and 96.29%, and epoch 3 reached 96.64% and 96.59%. By epoch 6 training accuracy was about 97.2% and validation accuracy about 96.8%. The validation loss was lowest at epoch 4 (about 0.080), rose slightly at epoch 5 and did not improve again, so the weights from the best epoch were restored and used for all results below.

The training and validation curves stay close together, which indicates that the small model learned general patterns instead of memorising the training URLs. The curves are shown in `reports/training_curves.png`.

6. Results

The final model was evaluated once on the untouched test set of 33,973 URLs.

Metric	Value
Accuracy	96.94%

Metric	Value
Precision (phishing)	97.24%
Recall (phishing)	96.62%
F1-score	96.93%
ROC-AUC	0.9965
Test loss	0.0756

Per-class report

Class	Precision	Recall	F1-score	Support
Legitimate	0.9665	0.9726	0.9696	17,000
Phishing	0.9724	0.9662	0.9693	16,973

Confusion matrix (rows = true class, columns = predicted class)

	Predicted legitimate	Predicted phishing
Actual legitimate	16,535	465
Actual phishing	573	16,400

Out of 17,000 legitimate URLs, 465 (2.7%) were wrongly flagged as phishing (false positives). Out of 16,973 phishing URLs, 573 (3.4%) were missed (false negatives). For a security tool, missed phishing URLs are the more costly error, and the recall of 96.62% reflects this. The decision threshold can be lowered if a deployment prefers to catch more phishing URLs at the price of more false alarms. The high ROC-AUC of 0.9965 shows that the model separates the two classes well across all possible thresholds.

6.1 Sanity check on hand-written URLs

Six URLs written by hand were tested after training. All were classified as expected. This is only a quick check and not a benchmark, because these examples are very typical of their class.

URL	Phishing probability	Result
https://www.wikipedia.org	0.005	Legitimate
https://www.google.com/search?q=python	0.000	Legitimate
https://github.com/tensorflow/tensorflow	0.007	Legitimate
http://paypal-secure-login.verify-account.example.ru/webscr.php?cmd=_login	1.000	Phishing
http://192.168.10.5/bank/login.php?id=8827364	0.997	Phishing
http://secure-update-amazon.account-verify.example.com/signin	1.000	Phishing

7. Deployment

7.1 Why the model was re-implemented in NumPy

The trained Keras model is tiny (about 240 KB), but TensorFlow itself is very large and does not fit comfortably into the size limits of a serverless function. Because the network consists only of an embedding lookup, one convolution, max pooling and two dense layers, its forward pass can be written in a few lines of NumPy. The trained weights were exported to `model_weights.npz`, and the inference code in `src/phishing_model.py` reproduces the model exactly.

This was verified against the original Keras model on 3,000 test URLs: the largest difference between the two probabilities was about 1.6e-06 and all predicted labels were identical. On those 3,000 URLs the NumPy version scored 97.13% accuracy.

7.2 System design

- **Frontend:** a static page (`public/index.html`) with a live demo, the results, dataset and code details, and links to the report, presentation, video and repository.
- **Backend:** a Python serverless function (`api/predict.py`) that accepts `POST /api/predict` with `{"url": "..."}` or `GET /api/predict?url=...` and returns the label and both probabilities as JSON. Empty or invalid input returns a clear error message.

- **Hosting:** the project is deployed on Vercel directly from the GitHub repository, so every update pushed to GitHub is redeployed automatically.

7.3 Testing

Automated tests (`tests/test_predict.py`) check the character encoding (fixed length, truncation, unknown characters), the output format, known legitimate and phishing URLs, input validation, accuracy on a 200-URL sample taken from the real test split, and agreement between the NumPy and Keras models. All 9 tests passed. The 200-URL sample gave 96.5% accuracy, consistent with the full test result.

8. Repository Contents

Path	Content
`api/predict.py`	Serverless prediction endpoint
`src/phishing_model.py`	NumPy inference code
`public/index.html`	Website and live demo
`artifacts/`	Trained model (`.keras`), weights (`.h5` and `.npz`), architecture, vocabulary, preprocessing settings, training history
`reports/`	Metrics file, training curves and confusion matrix
`tests/`	Unit tests and a 200-URL sample set
`notebooks/`	Google Colab training notebook
`README.md`	Overview, results, usage instructions and limitations

9. Limitations

1. **Only the URL text is used.** The model never opens the page and does not use domain age, certificates, page content or reputation lists, which a complete anti-phishing system would combine with it.

2. **Possible dataset shortcuts.** The phishing and legitimate URLs in the dataset may differ in style (for example `http` versus `https`, length or path structure). The model can learn such dataset-specific patterns, so accuracy on URLs collected from other sources may be lower than 96.94%.
3. **Changing attacks.** New attack styles, URL shorteners and unusual but legitimate URLs may be misclassified. The model is a snapshot of the data it was trained on and would need retraining as attacks change.
4. **Adversarial behaviour.** An attacker who knows the model could craft URLs that look harmless to it.
5. **No explanation.** The model gives a probability but not the reason for it.

Because of these limits, the output should be treated as one warning signal and not as the only security decision.

10. Conclusion and Future Work

This project showed that a very small character-level CNN with 17,633 parameters can classify URLs as phishing or legitimate with 96.94% accuracy and a ROC-AUC of 0.9965, using only the characters of the URL. The full workflow was completed: dataset selection and cleaning, leak-free splitting, model design, training with early stopping, honest test evaluation, export of the weights, automated tests, a public GitHub repository and a working web deployment that anyone can try.

Possible improvements are:

- Evaluate on a second, independent URL dataset to measure how well the model generalises beyond the training source.
- Compare with other architectures such as a bidirectional LSTM, a character-level Transformer, or a CNN with several kernel sizes.
- Add features that the URL text does not contain, for example domain age or page content.
- Tune the decision threshold for a lower miss rate, and add explanations by showing which parts of the URL influenced the prediction.
- Collect new phishing URLs regularly and retrain the model.

11. Project Links

Resource	Link
Live web application (Vercel)	[add Vercel link]
GitHub repository	[add GitHub link]
Presentation	[add presentation link]
YouTube video	[add YouTube link]

References

1. Dataset: `balanced_urls.csv`, Zenodo record 19371661, https://zenodo.org/records/19371661
2. TensorFlow and Keras documentation, https://www.tensorflow.org
3. Vercel Python functions documentation, https://vercel.com/docs